

```

/*Bismillahir-rahmanir rahim
Assalamualaikum
/* =====
Author    : tsi_sabit
Region    : bangladesh
Bangladesh university of business and technology
===== */

```

```

#include <bits/stdc++.h>
Using namespace std;

```

```

#define ll long long
#define int long long int
#define pb push_back
#define ppb pop_back
#define mp make_pair
#define fi first
#define se second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define yes cout << "yes" << endl;
#define no cout << "no" << endl;
#define nn "\n"
#define sz(x) (int)(x).size()
#define gcd(a, b) __gcd(a, b)
#define lcm(a, b) ((a) / gcd(a, b) * (b))
#define ms(arr, v) memset(arr, v, sizeof(arr))
#define rep(i, a, b) for (int i = a; i < b; i++)
#define rrep(i, a, b) for (int i = a; i >= b; i--)
#define rep0(i, n) for (int i = 0; i < n; i++)
#define repl(i, n) for (int i = 1; i <= n; i++)
#define read(x) int x; cin >> x;
#define readv(v, n) vector<int> v(n); for (auto &i : v) cin >> i;
#define setbit(x) __builtin_popcountll(x)
#define parity(x) __builtin_parityll(x)
#define clz(x) __builtin_clzll(x)
#define ctz(x) __builtin_ctzll(x)
#define unq(v) v.resize(distance(v.begin(), unique(v.begin(), v.end())));

```

```

#define pii pair<int, int>
#define pll pair<ll, ll>
#define vi vector<int>
#define vll vector<ll>
#define vpii vector<pii>
#define vpll vector<pll>

```

```

Const int mod = 1e9 + 7;
Const int inf = 1e9;
Const ll linf = 1e18;
Const double pi = acos(-1);

```

```

Void fastio() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
}

```

```

ll power(ll a, ll b, ll mod = mod) {
    ll res = 1;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

```

```

ll modinv(ll a, ll mod = mod) {
    return power(a, mod - 2, mod);
}

```

```

Int binaryasdecimal(int num) {
    int res = 0, base = 1;
    while (num > 0) {
        res += (num % 2) * base;
        base *= 10;
        num /= 2;
    }
    return res;
}

```

```

Bool ispoweroftwo(ll n) {
    return (n > 0 && (n & (n - 1)) == 0);
}

```

```

ll binarysearch(ll low, ll high, function<bool(ll)>
check) {
    while (low < high) {
        ll mid = (low + high) / 2;
        if (check(mid)) high = mid;
        else low = mid + 1;
    }
    return low;
}

```

```

Void vectorprint(vector<int>& v) {
    for (auto val : v) cout << val << ' ';
    cout << nn;
}

```

```

Bool issorted(const vector<int>& arr) {
    for (int i = 1; i < arr.size(); ++i) {
        if (arr[i] < arr[i - 1]) return false;
    }
    return true;
}

```

```

Bool issorteddescending(const vector<int>& arr) {
    for (int i = 1; i < (int)arr.size(); ++i) {
        if (arr[i] > arr[i - 1]) return false;
    }
    return true;
}

```

```

String inttobinary(int n) {
    if(n == 0) return "0";
    string res = "";
    while(n > 0) {
        res += (n % 2) + '0';
        n /= 2;
    }
    reverse(res.begin(), res.end());
    return res;
}

```

```

Vector<int> prefixsum(const vector<int>& v) {
    int n = v.size();
    vector<int> pre(n);
    pre[0] = v[0];
    for (int i = 1; i < n; i++) {
        pre[i] = pre[i - 1] + v[i];
    }
    return pre;
}

```

```

Vector<int> suffixsum(const vector<int>& v) {
    int n = v.size();
    vector<int> suf(n);
    suf[n - 1] = v[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        suf[i] = suf[i + 1] + v[i];
    }
}

```

```

    return suf;
}

Int arraygcd(const vector<int>& v) {
    int res = v[0];
    for (int i = 1; i < v.size(); i++) {
        res = gcd(res, v[i]);
        if (res == 1) break;
    }
    return res;
}

Int arraylcm(const vector<int>& v) {
    int res = v[0];
    for (int i = 1; i < v.size(); i++) {
        res = lcm(res, v[i]);
    }
    return res;
}

Map<int, int> frequencycount(const vector<int>& v) {
    map<int, int> freq;
    for (auto val : v) {
        freq[val]++;
    }
    return freq;
}

Map<char, int> charfrequency(const string& s) {
    map<char, int> freq;
    for (char c : s) {
        freq[c]++;
    }
    return freq;
}

Const int maxn = 1000000;
Int spf[maxn + 1];

Void sievespf() {
    for (int i = 1; i <= maxn; i++) spf[i] = i;
    for (int i = 2; i * i <= maxn; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j <= maxn; j += i) {
                if (spf[j] == j) spf[j] = i;
            }
        }
    }
}

Vector<int> getfactorization(int x) {
    vector<int> res;
    while (x != 1) {
        res.push_back(spf[x]);
        x /= spf[x];
    }
    return res;
}

Int sumofdivisors(int n) {
    int ans = 1;
    while (n > 1) {
        int p = spf[n];
        int cnt = 0;
        while (n % p == 0) {
            n /= p;
            cnt++;
        }
        int term = 1, power = 1;
        for (int i = 0; i < cnt; i++) {
            power *= p;
            term += power;
        }
    }
}

```

```

    }
    ans *= term;
}
return ans;
}

Int numofdivisors(int n) {
    int ans = 1;
    while (n > 1) {
        int p = spf[n];
        int cnt = 0;
        while (n % p == 0) {
            n /= p;
            cnt++;
        }
        ans *= (cnt + 1);
    }
    return ans;
}

Int productofdivisors(int n) {
    int d = numofdivisors(n);
    return power(n % mod, d / 2, mod);
}

Int mostfrequentelement(const vector<int>& v) {
    map<int, int> freq;
    int maxfreq = 0, mostfreq = v[0];
    for (int val : v) {
        freq[val]++;
        if (freq[val] > maxfreq) {
            maxfreq = freq[val];
            mostfreq = val;
        }
    }
    return mostfreq;
}

Const int n = 1e6 + 5;
Bool is_prime[n];
Void sieve(){
    fill(is_prime, is_prime + n, true);
    is_prime[0] = is_prime[1] = false;
    for(int i = 2; i * i < n; i++){
        if(is_prime[i]){
            for(int j = i * i; j < n; j += i){
                is_prime[j] = false;
            }
        }
    }
}

Ll factorial(ll n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}

Ll fact[maxn];

Void computefactorials(int n, ll mod = mod) {
    fact[0] = 1;
    for (int i = 1; i <= n; i++) {
        fact[i] = (fact[i - 1] * i) % mod;
    }
}

Vector<int> prefixminimum(const vector<int>& a) {
    int n = a.size();
    vector<int> pre(n);
    pre[0] = a[0];
    for (int i = 1; i < n; i++) {
        pre[i] = min(pre[i - 1], a[i]);
    }
}

```

```

    }
    return pre;
}

Vector<int> suffixminimum(const vector<int>& a) {
    int n = a.size();
    vector<int> suf(n);
    suf[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        suf[i] = min(suf[i + 1], a[i]);
    }
    return suf;
}

Vector<int> prefixmaximum(const vector<int>& a) {
    int n = a.size();
    vector<int> pre(n);
    pre[0] = a[0];
    for (int i = 1; i < n; i++) {
        pre[i] = max(pre[i - 1], a[i]);
    }
    return pre;
}

Vector<int> suffixmaximum(const vector<int>& a) {
    int n = a.size();
    vector<int> suf(n);
    suf[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        suf[i] = max(suf[i + 1], a[i]);
    }
    return suf;
}

Int phi[n];
Void computetotient() {
    for(int i = 0; i < n; i++) phi[i] = i;
    for(int i = 2; i < n; i++) {
        if(phi[i] == i) {
            for(int j = i; j < n; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
}

Int nthfibonacci(int n) {
    if (n == 1 || n == 2) return 1;
    int a = 1, b = 1, fib = 0;
    for (int i = 3; i <= n; i++) {
        fib = (a + b) % mod;
        a = b;
        b = fib;
    }
    return fib;
}

Char mostfrequentcharacter(const string& s) {
    map<char, int> freq;
    for(char c : s)
        freq[c]++;
    char ans = s[0];
    int mx = 0;
    for(pair<char, int> p : freq) {
        if(p.second > mx) {
            mx = p.second;
            ans = p.first;
        }
    }
    return ans;
}

Int lcslength(string a, string b) {
    int n = a.size();
    int m = b.size();

```

```

    vector<vector<int>> dp(n + 1, vector<int>(m + 1,
0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j -
1]);
            }
        }
    }
    return dp[n][m];
}

String lcs(string a, string b) {
    int n = a.size();
    int m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1,
0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j -
1]);
            }
        }
    }
    string ans = "";
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (a[i - 1] == b[j - 1]) {
            ans += a[i - 1];
            i--;
            j--;
        }
        else if (dp[i - 1][j] > dp[i][j - 1]) {
            i--;
        }
        else {
            j--;
        }
    }
    reverse(all(ans));
    return ans;
}

Int longestcommonsubstring(string a, string b) {
    int n = a.size();
    int m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1,
0));
    int ans = 0;
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            if(a[i - 1] == b[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
                ans = max(ans, dp[i][j]);
            }
        }
    }
    return ans;
}

Int editdistance(string a, string b) {
    int n = a.size();
    int m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for(int i = 0; i <= n; i++) dp[i][0] = i;
    for(int j = 0; j <= m; j++) dp[0][j] = j;
    for(int i = 1; i <= n; i++) {

```

```

    for(int j = 1; j <= m; j++) {
        if(a[i - 1] == b[j - 1])
            dp[i][j] = dp[i - 1][j - 1];
        else
            dp[i][j] = 1 + min({
                dp[i - 1][j],
                dp[i][j - 1],
                dp[i - 1][j - 1]
            });
    }
}
return dp[n][m];
}
Vector<int> buildlps(string pat) {
    int n = pat.size();
    vector<int> lps(n);
    int j = 0;
    for(int i = 1; i < n; i++) {
        while(j > 0 && pat[i] != pat[j])
            j = lps[j - 1];
        if(pat[i] == pat[j])
            j++;
        lps[i] = j;
    }
    return lps;
}
Vector<int> kmp(string text, string pat) {
    vector<int> lps = buildlps(pat);
    vector<int> pos;
    int j = 0;
    for(int i = 0; i < text.size(); i++) {
        while(j > 0 && text[i] != pat[j])
            j = lps[j - 1];
        if(text[i] == pat[j])
            j++;
        if(j == pat.size()) {
            pos.pb(i - pat.size() + 1);
            j = lps[j - 1];
        }
    }
    return pos;
}
Vector<int> zfunction(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i <= r)
            z[i] = min(r - i + 1, z[i - 1]);
        while(i + z[i] < n &&
            s[z[i]] == s[i + z[i]])
            z[i]++;
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
Struct dsu {
    vector<int> parent, sz;
    dsu(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        for(int i = 0; i <= n; i++)
            parent[i] = i;
    }
    int find(int v) {
        if(parent[v] == v)
            return v;
        return parent[v] =
            find(parent[v]);
    }
}

```

```

    }
    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if(a != b) {
            if(sz[a] < sz[b])
                swap(a, b);
            parent[b] = a;
            sz[a] += sz[b];
        }
    }
};
Vector<long long> dijkstra(int n,
    vector<vector<pair<int, int>>> &adj,
    int src)
{
    vector<long long> dist(n + 1, linf);
    priority_queue<pair<long long, int>,
        vector<pair<long long, int>>,
        greater<pair<long long, int>>> pq;
    dist[src] = 0;
    pq.push(make_pair(0ll, src));
    while (!pq.empty()) {
        long long d = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (d > dist[u])
            continue;
        for (int i = 0; i < (int)adj[u].size(); i++) {
            int v = adj[u][i].first;
            int w = adj[u][i].second;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push(make_pair(dist[v], v));
            }
        }
    }
    return dist;
}
String decimaltobasek(long long n, int base) {
    if (n == 0) return "0";
    string digits =
        "0123456789abcdefghijklmnopqrstuvwxyz";
    string res = "";
    while (n > 0) {
        res += digits[n % base];
        n /= base;
    }
    reverse(res.begin(), res.end());
    return res;
}
Long long basektodecimal(string s, int base) {
    long long value = 0;
    for (char c : s) {
        int digit;
        if ('0' <= c && c <= '9') digit = c - '0';
        else digit = c - 'a' + 10;
        value = value * base + digit;
    }
    return value;
}
String basetobase(string number, int frombase, int
    tobase) {
    long long decimal = basektodecimal(number,
        frombase);
    return decimaltobasek(decimal, tobase);
}
String convertbinarytocustom(string binary, char
    zerodigit, char onedigit) {
    for (char &c : binary) {
        if (c == '0') c = zerodigit;
    }
}

```

```

        else c = onedigit;
    }
    return binary;
}

String decimaltobinary(long long n) {
    string s;
    while (n > 0) {
        s += char('0' + (n % 2));
        n /= 2;
    }
    reverse(s.begin(), s.end());
    return s;
}

Void solve() {

}

Int32_t main() {
    fastio();
#ifdef local
    auto start =
chrono::high_resolution_clock::now();
#endif

    int t;
    cin >> t;
    while (t--) {
        solve();
    }

#ifdef local
    auto end = chrono::high_resolution_clock::now();
    cerr << "time: " <<
chrono::duration_cast<chrono::milliseconds>(end -
start).count() << " ms\n";
#endif

    return 0;
}

```